Docker - Volumes & Persistance

Un conteneur est éphémère, vos données ne devraient jamais l’être.

Frédéric Bellano

Pourquoi persister les données ?

Le problème fondamental

Un conteneur est éphémère par nature

- Les données écrites dans un conteneur sont **stockées dans une couche temporaire**
- À la suppression du conteneur, **tout disparaît**

Le problème fondamental

Un conteneur est éphémère par nature

- Les données écrites dans un conteneur sont **stockées dans une couche temporaire**
- À la suppression du conteneur, **tout disparaît**

Démonstration :

```
# Créer un fichier dans un conteneur
docker container run --name test alpine sh -c "echo 'Bonjour' > /data.txt"

# Supprimer le conteneur
docker container rm test

# Recréer un conteneur
docker container run --rm alpine cat /data.txt
# ❌ Erreur : le fichier n'existe plus
```

Le problème fondamental

Un conteneur est éphémère par nature

- Les données écrites dans un conteneur sont **stockées dans une couche temporaire**
- À la suppression du conteneur, **tout disparaît**

Démonstration :

```
# Créer un fichier dans un conteneur
docker container run --name test alpine sh -c "echo 'Bonjour' > /data.txt"

# Supprimer le conteneur
docker container rm test

# Recréer un conteneur
docker container run --rm alpine cat /data.txt
# ❌ Erreur : le fichier n'existe plus
```

⚠️ Sans mécanisme de persistance, impossible de conserver les données d'une base de données, les fichiers uploadés, les logs, etc.

Ce qu’on veut pouvoir faire

Ce qu’on veut pouvoir faire

- ✅ **Conserver** les données même si le conteneur est supprimé

Ce qu’on veut pouvoir faire

- ✅ **Conserver** les données même si le conteneur est supprimé
- ✅ **Partager** des fichiers entre plusieurs conteneurs

Ce qu’on veut pouvoir faire

- ✅ **Conserver** les données même si le conteneur est supprimé
- ✅ **Partager** des fichiers entre plusieurs conteneurs
- ✅ **Sauvegarder** et **restaurer** facilement

Ce qu’on veut pouvoir faire

- ✅ **Conserver** les données même si le conteneur est supprimé
- ✅ **Partager** des fichiers entre plusieurs conteneurs
- ✅ **Sauvegarder** et **restaurer** facilement
- ✅ **Découpler** les données de l’image et du conteneur

Ce qu’on veut pouvoir faire

- ✅ **Conserver** les données même si le conteneur est supprimé
- ✅ **Partager** des fichiers entre plusieurs conteneurs
- ✅ **Sauvegarder** et **restaurer** facilement
- ✅ **Découpler** les données de l’image et du conteneur

> 💡 Docker propose **3 mécanismes** pour répondre à ces besoins.

Les 3 types de stockage

Les 3 types de stockage

Mapping de FS

Volumes

FS en RAM

Mapping de système de fichiers

Bind mounts

Partage d'un **répertoire** de votre système hôte avec un conteneur Docker.

Mapping de système de fichiers

Bind mounts

Partage d'un **répertoire** de votre système hôte avec un conteneur Docker.

- Utile pour partager des fichiers ou des données de configuration entre l'hôte et le conteneur
- Données stockées sur le système hôte, indépendamment du cycle de vie du conteneur

Mapping de système de fichiers

Bind mounts

Partage d'un **répertoire** de votre système hôte avec un conteneur Docker.

- Utile pour partager des fichiers ou des données de configuration entre l'hôte et le conteneur
- Données stockées sur le système hôte, indépendamment du cycle de vie du conteneur

```
docker container run -v /chemin/du/systeme/hote:/chemin/dans/le/conteneur mon_image
```

Volumes

Stockage persistant ⇒ données en dehors du système de fichiers du conteneur.

Volumes

Stockage persistant ⇒ données en dehors du système de fichiers du conteneur.

- Utile pour stocker des données qui doivent survivre à l'arrêt ou à la suppression d'un conteneur.
- Volumes indépendants du cycle de vie du conteneur.

Volumes

Stockage persistant ⇒ données en dehors du système de fichiers du conteneur.

- Utile pour stocker des données qui doivent survivre à l'arrêt ou à la suppression d'un conteneur.
- Volumes indépendants du cycle de vie du conteneur.

```
docker container run -v nom_du_volume:/chemin/dans/le/conteneur
```

FS en RAM

Système de fichiers temporaire stocké en RAM.

FS en RAM

Système de fichiers temporaire stocké en RAM.

- Utile lorsque des données temporaires doivent être stockées en mémoire pour des performances élevées.
- Les données n’existent que tant que le conteneur est en cours d’exécution.

FS en RAM

Système de fichiers temporaire stocké en RAM.

- Utile lorsque des données temporaires doivent être stockées en mémoire pour des performances élevées.
- Les données n’existent que tant que le conteneur est en cours d’exécution.

```
docker container run --tmpfs /chemin/dans/le/conteneur
```

FS en RAM

Système de fichiers temporaire stocké en RAM.

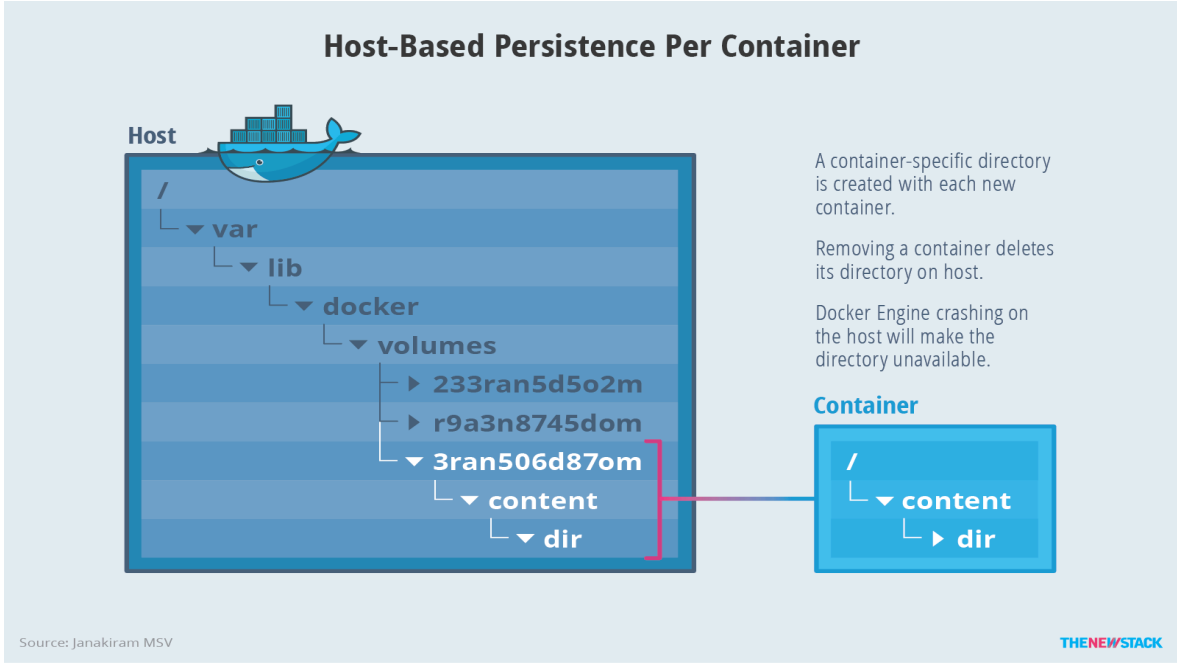
- Utile lorsque des données temporaires doivent être stockées en mémoire pour des performances élevées.
- Les données n'existent que tant que le conteneur est en cours d'exécution.

```
docker container run --tmpfs /chemin/dans/le/conteneur
```

Option clé : `--tmpfs`

Détail de chaque système

Mapping de système de fichiers



Mapping de système de fichiers

Partage de dossier

```
-v /some/content:/usr/share/nginx/html:ro
```

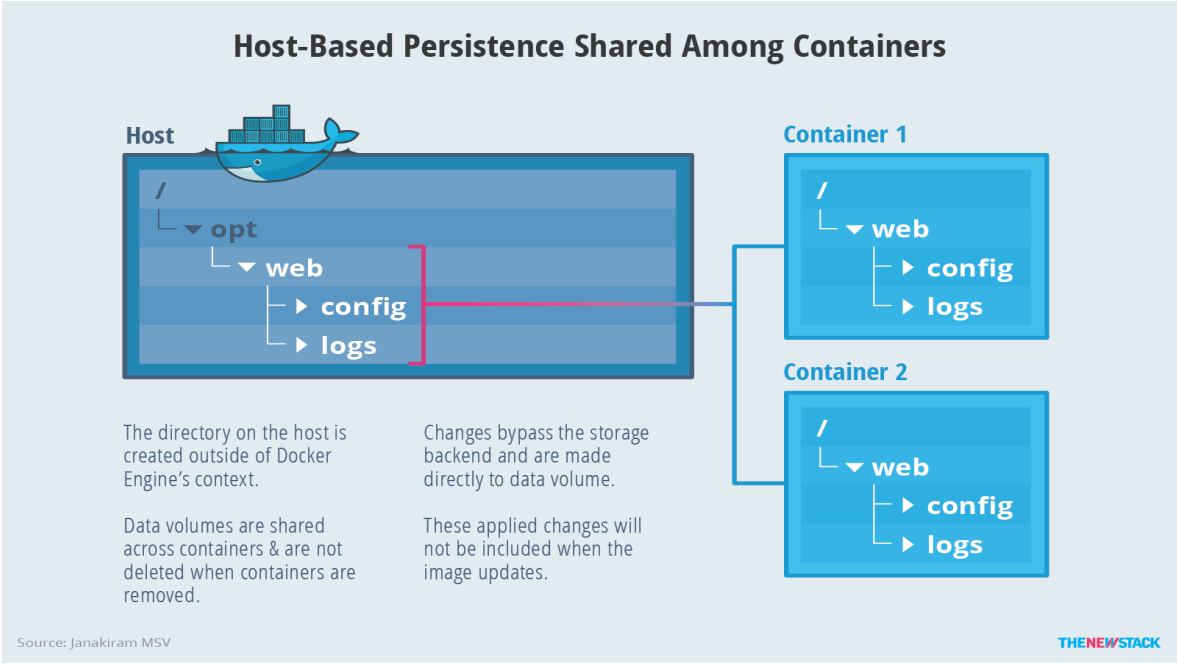
Chemin machine hôte	Chemin conteneur	Lecture seule
/some/content	/usr/share/nginx/html	oui

Partage de fichier

```
-v ~/.bash_history:/.bash_history
```

Chemin machine hôte	Chemin conteneur	Lecture seule
~/.bash_history	/.bash_history	non

Volumes Docker



Les volumes

Il est possible de créer des "espaces mémoire" que l’on peut mettre à disposition des conteneurs.

Les volumes

Il est possible de créer des "espaces mémoire" que l’on peut mettre à disposition des conteneurs.

```
docker volume create --name my-data
docker container run -v my-data:/data busybox ls /data
```

Les volumes

Il est possible de créer des "espaces mémoire" que l'on peut mettre à disposition des conteneurs.

```
docker volume create --name my-data
docker container run -v my-data:/data busybox ls /data
```

> **my-data** n'est plus un chemin absolu sur l'hôte, mais juste le nom du volume géré par Docker.

Les volumes

Commandes utiles

Les volumes

Commandes utiles

Lister tous les volumes

Les volumes

Commandes utiles

Lister tous les volumes

```
docker volume ls
```

Les volumes

Commandes utiles

Lister tous les volumes

```
docker volume ls
```

Supprimer un volume

Les volumes

Commandes utiles

Lister tous les volumes

```
docker volume ls
```

Supprimer un volume

```
docker volume rm my-data
```

Les volumes

Commandes utiles

Lister tous les volumes

```
docker volume ls
```

Supprimer un volume

```
docker volume rm my-data
```

Inspecter un volume

Les volumes

Commandes utiles

Lister tous les volumes

```
docker volume ls
```

Supprimer un volume

```
docker volume rm my-data
```

Inspecter un volume

```
docker volume inspect my-data
```

Les volumes

Commandes utiles

Lister tous les volumes

```
docker volume ls
```

Supprimer un volume

```
docker volume rm my-data
```

Inspecter un volume

```
docker volume inspect my-data
```

Supprimer un volume

Les volumes

Commandes utiles

Lister tous les volumes

```
docker volume ls
```

Supprimer un volume

```
docker volume rm my-data
```

Inspecter un volume

```
docker volume inspect my-data
```

Supprimer un volume

```
docker volume rm my-data
```

Les volumes

Commandes utiles

Lister tous les volumes

```
docker volume ls
```

Supprimer un volume

```
docker volume rm my-data
```

Inspecter un volume

```
docker volume inspect my-data
```

Supprimer un volume

```
docker volume rm my-data
```

Supprimer tous les volumes inutilisés

Les volumes

Commandes utiles

Lister tous les volumes

```
docker volume ls
```

Supprimer un volume

```
docker volume rm my-data
```

Inspecter un volume

```
docker volume inspect my-data
```

Supprimer un volume

```
docker volume rm my-data
```

Supprimer tous les volumes inutilisés

```
docker volume prune
```

FileSystem en RAM

```
docker container run
  -d
  --read-only
  --tmpfs /tmp
  nginx
```

> Les données sont perdues à l'arrêt du conteneur, mais très rapides à lire/écrire.

FileSystem en RAM

```
docker container run
  -d
  --read-only
  --tmpfs /tmp
  nginx
```

> Les données sont perdues à l'arrêt du conteneur, mais très rapides à lire/écrire.

`read-only` Le conteneur ne peut pas écrire sur le FS.

FileSystem en RAM

```
docker container run
  -d
  --read-only
  --tmpfs /tmp
  nginx
```

> Les données sont perdues à l'arrêt du conteneur, mais très rapides à lire/écrire.

`read-only` Le conteneur ne peut pas écrire sur le FS.

`--tmpfs` Sauf là.

FileSystem en RAM

```
docker container run
  -d
  --read-only
  --tmpfs /tmp
  nginx
```

> Les données sont perdues à l'arrêt du conteneur, mais très rapides à lire/écrire.

`read-only` Le conteneur ne peut pas écrire sur le FS.

`--tmpfs` Sauf là.